Firmas de anillo

Del esquema a la implementación en Python

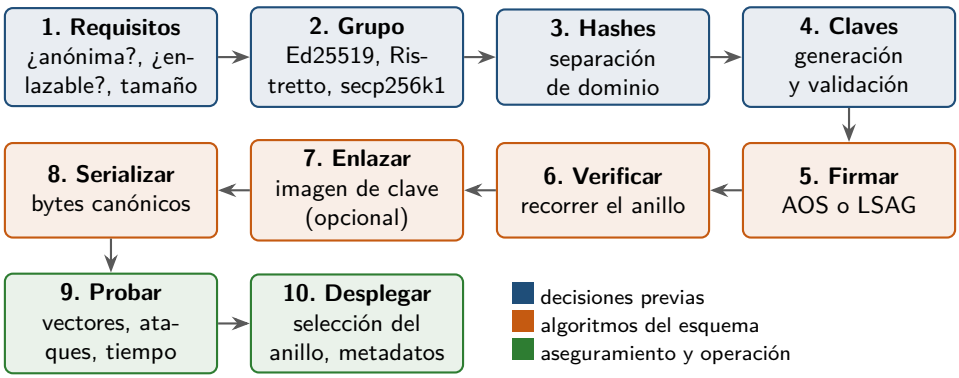
Dr. José de Jesús Ángel Ángel

De qué trata este documento

Una firma de anillo permite que un miembro de un conjunto de claves públicas $L = (Y_1, \dots, Y_n)$ firme un mensaje de modo que cualquiera pueda verificar que la firmó *alguien* de L, sin poder saber quién. Aquí se recorre el camino completo para implementarla: requisitos, elección del grupo, hashes, algoritmos en pseudocódigo, una implementación de referencia en Python que funciona, pruebas, errores típicos y, al final, aplicaciones reales y escenarios de uso.

1 El pipeline de un vistazo

La implementación se organiza en diez etapas. Las cuatro primeras son decisiones que se toman antes de escribir código; las de en medio son los algoritmos; las últimas aseguran que lo construido es correcto y seguro.



2 Notación

En los documentos anteriores trabajamos en $\mathbb{Z}_p^*$ con notación multiplicativa. En la práctica se usan curvas elípticas, donde la operación del grupo se escribe como suma. La traducción es directa:

Concepto	Notación multiplicativa ($\mathbb{Z}_p^*$)	Notación aditiva (curva elíptica)
Generador y orden	g de orden q	punto G de orden primo ℓ
Clave pública	$y_i = g^{x_i}$	$Y_i = x_i G$
Compromiso	$R = g^a$	$R = aG$
Ecuación del eslabón	$R_i = g^{r_i} y_i^{c_i}$	$R_i = r_i G + c_i Y_i$
Cierre	$r_s = a - x_s c_s \bmod q$	$r_s = a - c_s x_s \bmod \ell$

Además usaremos:

- $H_{\mathrm{s}} : \{0,1\}^* \to \mathbb{Z}_\ell$, un hash a escalar, y $H_{\mathrm{p}} : \{0,1\}^* \to \langle G \rangle$, un hash a punto.
- $\langle \cdot \rangle$ para la codificación en bytes, canónica e inyectiva, de puntos, escalares y listas.

- $x \xleftarrow{\$} \mathbb{Z}_\ell$ para «elegir x uniformemente al azar» con un generador criptográfico.
- Índices cíclicos: después de n viene 1. En el código, que cuenta desde 0, después de $n-1$ viene 0.

3 Etapas previas: decisiones antes del código

3.1 Etapa 1. Requisitos

Decisión de diseño

- **¿Solo anonimato, o también enlazabilidad?** Si el sistema debe impedir que alguien firme dos veces (votos, gastos de una criptomoneda), se necesita una firma *enlazable* como LSAG. Si solo se quiere probar pertenencia sin revelar identidad (una denuncia), basta AOS.
- **Tamaño del anillo.** El anonimato máximo es $1/n$, pero el tamaño de la firma y el tiempo de verificación crecen linealmente con n.
- **¿Quién forma el anillo?** Las claves públicas deben existir antes y ser auténticas: un directorio institucional, claves SSH o PGP ya publicadas, o salidas de una cadena de bloques.

Esquema	Qué ofrece	Tamaño de la firma	Cuándo usarlo
RST (2001) [1]	Anonimato; claves RSA	n valores más un sello	Interés histórico; claves RSA existentes
AOS (2002) [2]	Anonimato; claves de logaritmo discreto	$n+1$ escalares	Pertenencia anónima sin control de repetición
LSAG (2004) [3]	Anonimato + enlazabilidad	$n+1$ escalares + 1 punto	Votación, evitar dobles gastos
CLSAG (2019) [7]	LSAG compacto con varias claves por miembro	$n+1$ escalares + puntos de imagen	Monero (RingCT)
Groth–Kohlweiss (2015) [4]	Anonimato con firma pequeña	$O(\log n)$	Anillos muy grandes

3.2 Etapa 2. Elección del grupo

El grupo debe tener orden primo ℓ de unos 256 bits (seguridad de unos 128 bits) y aritmética eficiente y bien implementada. Las opciones habituales:

- **Ed25519 (Curve25519 en forma de Edwards).** Rápida y disponible en libsodium. Su grupo completo tiene cofactor 8: hay que trabajar en el subgrupo primo y rechazar puntos de orden pequeño. Es la que usa Monero.
- **Ristretto255.** Una capa sobre Ed25519 que elimina el cofactor y da un grupo de orden primo con codificación canónica. Es la opción más limpia para protocolos nuevos.
- **secp256k1.** La curva de Bitcoin, de orden primo, disponible en libsecp256k1.

Cuidado

Los grupos de juguete de los documentos anteriores ($p = 23$, $q = 11$) sirven para entender, nunca para proteger nada: con $q = 11$ cualquiera falsifica una firma probando todas las opciones. Tampoco conviene escribir la aritmética de curva uno mismo: se usa una biblioteca probada.

3.3 Etapa 3. Funciones hash

Las funciones H_{s} y H_{p} se construyen a partir de un hash criptográfico como SHA-512 o BLAKE2b, con tres cuidados:

1. **Separación de dominio.** Cada uso lleva una etiqueta distinta, como `"anillo/AOS/v1"` o `"anillo/LSAG/v1"`, para que un hash calculado en un contexto no sirva en otro.
2. **Codificación inyectiva.** Cada pieza se precede de su longitud; así (`"ab"`, `"c"`) y (`"a"`, `"bc"`) no producen la misma entrada.
3. **Reducción amplia.** Para obtener un escalar uniforme en $\mathbb{Z}_\ell$ se reducen 64 bytes de salida módulo ℓ, no 32: con 32 bytes la distribución queda sesgada.

Lo que entra al hash debe incluir el **anillo completo** L, el mensaje m y, en LSAG, la imagen de clave I:

$$c_{i+1} = H_{\mathrm{s}}(\texttt{dominio} \,\|\, \langle L \rangle \,\|\, \langle m \rangle \,\|\, \langle R_i \rangle).$$

Incluir L impide que una firma válida para un anillo se presente como válida para otro.

3.4 Etapa 4. Claves

Algoritmo 1 Generación de claves

Entrada: parámetros del grupo (G, ℓ)
Salida: par (x, Y)

1: $x \xleftarrow{\$} \mathbb{Z}_\ell \setminus \{0\}$ ▷ generador criptográfico: `secrets`
2: $Y \leftarrow xG$
3: **devolver** (x, Y) ▷ x se guarda en secreto; Y se publica

Al *recibir* claves ajenas para formar un anillo, cada Y_i se valida: debe ser un punto de la curva, pertenecer al subgrupo de orden ℓ y no ser el neutro. Además, el anillo debe quedar en un **orden canónico** (por ejemplo, ordenando las codificaciones), para que firmante y verificador usen exactamente la misma lista L.

4 Los algoritmos del esquema

4.1 Etapa 5 y 6. AOS: firmar y verificar

El esquema de Abe, Ohkubo y Suzuki [2] es la cadena de desafíos que vimos en los documentos visuales, ahora en notación aditiva y con el hash completo.

Algoritmo 2 Firma AOS

Entrada: mensaje m; anillo $L = (Y_0, \dots, Y_{n-1})$; índice s; clave secreta x_s con $Y_s = x_s G$
Salida: firma $\sigma = (c_0; r_0, \dots, r_{n-1})$

$a \xleftarrow{\$} \mathbb{Z}_\ell$ ▷ aleatorio secreto, nuevo en cada firma
$c_{s+1} \leftarrow H_{\mathrm{s}}(\texttt{AOS} \,\|\, \langle L \rangle \,\|\, \langle m \rangle \,\|\, \langle aG \rangle)$
$i \leftarrow s + 1 \bmod n$
mientras $i \neq s$ **hacer** ▷ simular a los demás miembros
    $r_i \xleftarrow{\$} \mathbb{Z}_\ell$
    $R_i \leftarrow r_i G + c_i Y_i$
    $c_{i+1} \leftarrow H_{\mathrm{s}}(\texttt{AOS} \,\|\, \langle L \rangle \,\|\, \langle m \rangle \,\|\, \langle R_i \rangle)$
    $i \leftarrow i + 1 \bmod n$
fin mientras
$r_s \leftarrow a - c_s\, x_s \bmod \ell$ ▷ cerrar el anillo con la clave
devolver $(c_0; r_0, \dots, r_{n-1})$

Algoritmo 3 Verificación AOS

Entrada: mensaje m; anillo L; firma $\sigma = (c_0; r_0, \dots, r_{n-1})$
Salida: válida o inválida

si algún Y_i no es un punto válido del subgrupo, o algún escalar no es canónico ($\geq \ell$) **entonces**
    devolver inválida
fin si
$c \leftarrow c_0$
para $i = 0, 1, \dots, n-1$ **hacer**
    $R_i \leftarrow r_i G + c\, Y_i$
    $c \leftarrow H_{\mathrm{s}}(\texttt{AOS} \,\|\, \langle L \rangle \,\|\, \langle m \rangle \,\|\, \langle R_i \rangle)$
fin para
devolver válida si $c = c_0$ (comparación en tiempo constante); si no, inválida

Corrección. En el nodo del firmante el verificador recalcula

$$r_s G + c_s Y_s = (a - c_s x_s) G + c_s (x_s G) = aG,$$

que es justo el compromiso usado al firmar; en los demás nodos R_i se definió con esa misma fórmula. Por lo tanto, todos los desafíos se reproducen y la vuelta regresa a c_0.

4.2 Etapa 7. LSAG: la versión enlazable

LSAG [3] agrega una **imagen de clave** $I = x_s H_{\mathrm{p}}(Y_s)$. Es determinista: el mismo firmante produce siempre el mismo I, sin importar el mensaje ni el anillo, pero I no revela cuál de las Y_i es la suya. Cada eslabón demuestra a la vez dos relaciones con el mismo r_i: una respecto de G y otra respecto de $H_{\mathrm{p}}(Y_i)$.

Algoritmo 4 Firma LSAG

Entrada: m; $L = (Y_0, \dots, Y_{n-1})$; índice s; clave x_s
Salida: $\sigma = (c_0; r_0, \dots, r_{n-1}; I)$

$I \leftarrow x_s\, H_{\mathrm{p}}(Y_s)$ ▷ imagen de clave
$a \xleftarrow{\$} \mathbb{Z}_\ell$
$c_{s+1} \leftarrow H_{\mathrm{s}}(\mathtt{LSAG} \,\|\, \langle L\rangle \,\|\, \langle I\rangle \,\|\, \langle m\rangle \,\|\, \langle aG\rangle \,\|\, \langle aH_{\mathrm{p}}(Y_s)\rangle)$
$i \leftarrow s + 1 \bmod n$
mientras $i \neq s$ **hacer**
    $r_i \xleftarrow{\$} \mathbb{Z}_\ell$
    $U_i \leftarrow r_i G + c_i Y_i; \quad V_i \leftarrow r_i H_{\mathrm{p}}(Y_i) + c_i I$
    $c_{i+1} \leftarrow H_{\mathrm{s}}(\mathtt{LSAG} \,\|\, \langle L\rangle \,\|\, \langle I\rangle \,\|\, \langle m\rangle \,\|\, \langle U_i\rangle \,\|\, \langle V_i\rangle)$
    $i \leftarrow i + 1 \bmod n$
fin mientras
$r_s \leftarrow a - c_s\, x_s \bmod \ell$
devolver $(c_0; r_0, \dots, r_{n-1}; I)$

Algoritmo 5 Verificación LSAG y enlace

función VerificarLSAG(m, L, σ)
    validar los Y_i, la imagen I (punto del subgrupo, no neutro) y los escalares
    $c \leftarrow c_0$
    para $i = 0, \dots, n-1$ **hacer**
        $U_i \leftarrow r_i G + c Y_i; \quad V_i \leftarrow r_i H_{\mathrm{p}}(Y_i) + c I$
        $c \leftarrow H_{\mathrm{s}}(\mathtt{LSAG} \,\|\, \langle L\rangle \,\|\, \langle I\rangle \,\|\, \langle m\rangle \,\|\, \langle U_i\rangle \,\|\, \langle V_i\rangle)$
    fin para
    devolver $c = c_0$
fin función
función Enlazadas(σ, σ') ▷ ambas ya verificadas
    devolver $I = I'$ ▷ misma imagen ⇒ mismo firmante
fin función

Por qué la imagen de clave no rompe el anonimato

En el nodo del firmante, $V_s = r_s H_{\mathrm{p}}(Y_s) + c_s I = (a - c_s x_s) H_{\mathrm{p}}(Y_s) + c_s x_s H_{\mathrm{p}}(Y_s) = a H_{\mathrm{p}}(Y_s)$, así que la vuelta cierra. Para distinguir a qué Y_i corresponde I, un observador tendría que decidir si $\log_G Y_i = \log_{H_{\mathrm{p}}(Y_i)} I$, un problema tipo Diffie–Hellman decisional que se supone difícil. Así, I funciona como un «número de folio» anónimo: permite detectar a quien firma dos veces sin saber quién es.

4.3 Etapa 8. Serialización

La firma viaja como bytes. Con puntos y escalares de 32 bytes:

Esquema	Contenido	Tamaño
AOS	$c_0 \,\|\, r_0 \,\|\, \cdots \,\|\, r_{n-1}$	$32(n+1)$ bytes
LSAG	$c_0 \,\|\, r_0 \,\|\, \cdots \,\|\, r_{n-1} \,\|\, I$	$32(n+2)$ bytes

Con $n = 16$, una firma LSAG ocupa 576 bytes. El verificador necesita además el anillo L ($32n$ bytes), que suele referirse por identificadores en vez de transmitirse completo. Al deserializar se rechaza todo escalar $\geq \ell$: si se aceptaran, la misma firma tendría varias codificaciones válidas (*maleabilidad*).

5 Bibliotecas de Python

Python no trae aritmética de curvas elípticas en la biblioteca estándar, pero sí lo necesario para hashes y aleatoriedad. Lo que hace falta es una biblioteca que exponga operaciones de *bajo nivel*: sumar puntos y multiplicar un punto por un escalar.

Biblioteca	Grupo	Qué aporta al pipeline
`hashlib` (estándar)	—	SHA-256, SHA-512, SHA-3, BLAKE2 para construir H_{s} y H_{p}.
`secrets` (estándar)	—	Aleatoriedad criptográfica (`token_bytes`) y `compare_digest` para comparar en tiempo constante.
`PyNaCl` (libsodium)	Ed25519	Suma de puntos, multiplicación sin «clamping», reducción y aritmética de escalares módulo ℓ, hash a punto (`from_uniform`) y validación de puntos. Es la base de la implementación de referencia de la sección 6.
`coincurve` (libsecp256k1)	secp256k1	Enlaces a la biblioteca de Bitcoin. `PublicKey.multiply`, `PublicKey.combine_keys` (suma de puntos) y `PrivateKey.add/multiply` para escalares.
`ecdsa` (python-ecdsa)	secp256k1, NIST P-256 y otras	Python puro; puntos con suma y multiplicación por escalar. Cómoda para prototipos y docencia; no es de tiempo constante.
`fastecdsa`	secp256k1, P-256, …	Aritmética de puntos en C con operadores `+` y `*`.
`py_ecc`	secp256k1, BN128, BLS12-381	Python puro, del ecosistema Ethereum; útil para experimentar.
`pycryptodome`	curvas NIST, Ed25519	Objetos `EccPoint` con aritmética de puntos.
`sympy`, `gmpy2`	$\mathbb{Z}_p^*$	Primos, órdenes y potencias modulares rápidas para versiones de juguete en clase.

Cuidado

- La popular biblioteca `cryptography` (pyca) ofrece firmas Ed25519 y ECDSA completas, pero **no** expone la suma de puntos ni la multiplicación por escalares arbitrarios, así que no sirve para construir firmas de anillo.
- Las bibliotecas de Python puro no son de tiempo constante: el tiempo de cálculo puede filtrar información sobre la clave secreta. Para producción conviene una biblioteca en C o Rust auditada.
- En GitHub hay implementaciones educativas de firmas de anillo en Python; úsalas para estudiar, no para proteger datos reales.

6 Implementación de referencia

El siguiente programa implementa AOS y LSAG sobre Ed25519 con PyNaCl. Se probó con PyNaCl 1.6.2: firma y verifica en todas las posiciones del anillo y con $n = 2, 3, 8$; rechaza firmas manipu-

ladas y anillos reordenados; y enlaza correctamente dos firmas LSAG del mismo firmante. Instalación: `pip install pynacl`.

```python
"""Firmas de anillo AOS y LSAG sobre Ed25519 (PyNaCl / libsodium).

Implementación de referencia con fines didácticos: no está auditada.
"""
import hashlib
import secrets
from nacl import bindings as nb

L_BYTES = 32  # tamaño de puntos y escalares codificados


# ---------- Primitivas del grupo ----------
def escalar_aleatorio() -> bytes:
    """Escalar uniforme en Z_l: 64 bytes aleatorios reducidos (sin sesgo)."""
    return nb.crypto_core_ed25519_scalar_reduce(secrets.token_bytes(64))


def mul_base(k: bytes) -> bytes:          # k·G
    return nb.crypto_scalarmult_ed25519_base_noclamp(k)


def mul(k: bytes, P: bytes) -> bytes:     # k·P
    return nb.crypto_scalarmult_ed25519_noclamp(k, P)


def suma(P: bytes, Q: bytes) -> bytes:    # P + Q
    return nb.crypto_core_ed25519_add(P, Q)


def es_punto_valido(P: bytes) -> bool:
    return len(P) == L_BYTES and nb.crypto_core_ed25519_is_valid_point(P)


def es_escalar_canonico(k: bytes) -> bool:
    """k < l: reducir no debe cambiarlo."""
    return len(k) == L_BYTES and nb.crypto_core_ed25519_scalar_reduce(k + bytes(32)) == k


# ---------- Hashes con separación de dominio ----------
def _codifica(*partes: bytes) -> bytes:
    """Concatena con prefijo de longitud para que la codificación sea inyectiva."""
    return b"".join(len(p).to_bytes(8, "big") + p for p in partes)


def H_escalar(dominio: bytes, *partes: bytes) -> bytes:
    h = hashlib.sha512(_codifica(dominio, *partes)).digest()
    return nb.crypto_core_ed25519_scalar_reduce(h)


def H_punto(Y: bytes) -> bytes:
    """Hash a un punto del subgrupo primo (para la imagen de clave de LSAG)."""
    h = hashlib.sha512(_codifica(b"anillo/H_punto/v1", Y)).digest()[:32]
    return nb.crypto_core_ed25519_from_uniform(h)


# ---------- Claves ----------
def generar_claves():
```

```
    x = escalar_aleatorio()
    return x, mul_base(x)


def anillo_bytes(anillo) -> bytes:
    return b"".join(anillo)


# ---------- AOS (Abe-Ohkubo-Suzuki) ----------
def firmar_aos(m: bytes, anillo: list, s: int, x_s: bytes):
    n = len(anillo)
    assert anillo[s] == mul_base(x_s), "la clave secreta no corresponde a la posición s"
    Lb = anillo_bytes(anillo)
    c = [None] * n
    r = [None] * n
    a = escalar_aleatorio()
    c[(s + 1) % n] = H_escalar(b"anillo/AOS/v1", Lb, m, mul_base(a))
    i = (s + 1) % n
    while i != s:
        r[i] = escalar_aleatorio()
        R_i = suma(mul_base(r[i]), mul(c[i], anillo[i]))
        c[(i + 1) % n] = H_escalar(b"anillo/AOS/v1", Lb, m, R_i)
        i = (i + 1) % n
    r[s] = nb.crypto_core_ed25519_scalar_sub(a, nb.crypto_core_ed25519_scalar_mul(c[s], x_s))
    return c[0], r


def verificar_aos(m: bytes, anillo: list, firma) -> bool:
    c0, r = firma
    n = len(anillo)
    if len(r) != n or not all(map(es_punto_valido, anillo)):
        return False
    if not es_escalar_canonico(c0) or not all(map(es_escalar_canonico, r)):
        return False
    Lb = anillo_bytes(anillo)
    c = c0
    for i in range(n):
        R_i = suma(mul_base(r[i]), mul(c, anillo[i]))
        c = H_escalar(b"anillo/AOS/v1", Lb, m, R_i)
    return secrets.compare_digest(c, c0)


# ---------- LSAG (Liu-Wei-Wong): enlazable ----------
def firmar_lsag(m: bytes, anillo: list, s: int, x_s: bytes):
    n = len(anillo)
    Lb = anillo_bytes(anillo)
    I = mul(x_s, H_punto(anillo[s]))                  # imagen de clave
    c = [None] * n
    r = [None] * n
    a = escalar_aleatorio()
    c[(s + 1) % n] = H_escalar(b"anillo/LSAG/v1", Lb, I, m,
                               mul_base(a), mul(a, H_punto(anillo[s])))
    i = (s + 1) % n
    while i != s:
        r[i] = escalar_aleatorio()
        U = suma(mul_base(r[i]), mul(c[i], anillo[i]))
        V = suma(mul(r[i], H_punto(anillo[i])), mul(c[i], I))
        c[(i + 1) % n] = H_escalar(b"anillo/LSAG/v1", Lb, I, m, U, V)
        i = (i + 1) % n
    r[s] = nb.crypto_core_ed25519_scalar_sub(a, nb.crypto_core_ed25519_scalar_mul(c[s], x_s))
```

```
    return c[0], r, I


def verificar_lsag(m: bytes, anillo: list, firma) -> bool:
    c0, r, I = firma
    n = len(anillo)
    if len(r) != n or not es_punto_valido(I) or not all(map(es_punto_valido, anillo)):
        return False
    if not es_escalar_canonico(c0) or not all(map(es_escalar_canonico, r)):
        return False
    Lb = anillo_bytes(anillo)
    c = c0
    for i in range(n):
        U = suma(mul_base(r[i]), mul(c, anillo[i]))
        V = suma(mul(r[i], H_punto(anillo[i])), mul(c, I))
        c = H_escalar(b"anillo/LSAG/v1", Lb, I, m, U, V)
    return secrets.compare_digest(c, c0)


def enlazadas(f1, f2) -> bool:
    """Dos firmas LSAG válidas con la misma imagen de clave vienen del mismo firmante."""
    return f1[2] == f2[2]


# ---------- Demostración ----------
if __name__ == "__main__":
    claves = [generar_claves() for _ in range(5)]
    anillo = [Y for _, Y in claves]
    s = 3
    m = "Informe confidencial del comité".encode()

    sigma = firmar_aos(m, anillo, s, claves[s][0])
    print("AOS válida:", verificar_aos(m, anillo, sigma))
    print("AOS con mensaje alterado:", verificar_aos(b"otro", anillo, sigma))

    f1 = firmar_lsag(m, anillo, s, claves[s][0])
    f2 = firmar_lsag(b"segundo voto", anillo, s, claves[s][0])
    f3 = firmar_lsag(b"tercer voto", anillo, 1, claves[1][0])
    print("LSAG válidas:", verificar_lsag(m, anillo, f1), verificar_lsag(b"segundo voto", anillo,
    f2))
    print("f1 y f2 enlazadas (mismo firmante):", enlazadas(f1, f2))
    print("f1 y f3 enlazadas (distinto firmante):", enlazadas(f1, f3))
```

Salida del programa

```
AOS válida: True
AOS con mensaje alterado: False
LSAG válidas: True True
f1 y f2 enlazadas (mismo firmante): True
f1 y f3 enlazadas (distinto firmante): False
```

En una máquina de nube típica, con $n = 16$, firmar y verificar LSAG toma unos 5 ms cada uno, y la firma ocupa 576 bytes.

Correspondencia con los algoritmos. `generar_claves` es el algoritmo 1; `firmar_aos` y `verificar_aos` son los algoritmos 2 y 3; `firmar_lsag`, `verificar_lsag` y `enlazadas` son los algoritmos 4 y 5. La función `_codifica` implementa la codificación inyectiva de la etapa 3, y

`es_escalar_canonico` la regla contra la maleabilidad de la etapa 8.

7 Etapa 9. Pruebas

Prueba	Qué comprueba
Ida y vuelta	Para todo n y toda posición s, $\mathsf{Verificar}(m, L, \mathsf{Firmar}(m, L, s, x_s)) = \mathsf{válida}$.
Manipulación	Cambiar m, cualquier r_i, c_0, I o el orden de L produce $\mathsf{inválida}$.
Vectores fijos	Con aleatoriedad fija, la firma coincide byte a byte con valores guardados; detecta cambios accidentales en la codificación.
Entradas hostiles	Puntos fuera de la curva, de orden pequeño o neutros; escalares $\geq \ell$; longitudes incorrectas. Todo se rechaza sin excepciones no controladas.
Enlace	Dos firmas LSAG del mismo firmante comparten I; de firmantes distintos, no.
Anonimato (estadística)	Con muchas firmas de firmantes distintos, ninguna componente de σ se correlaciona con s.
Rendimiento	Tiempo de firma y verificación frente a n; debe crecer de forma lineal.

Errores que rompen el esquema

1. **Reutilizar el aleatorio a.** Si el mismo firmante usa el mismo a en dos firmas con desafíos $c_s \neq c'_s$, entonces $r_s = a - c_s x_s$ y $r'_s = a - c'_s x_s$, de donde

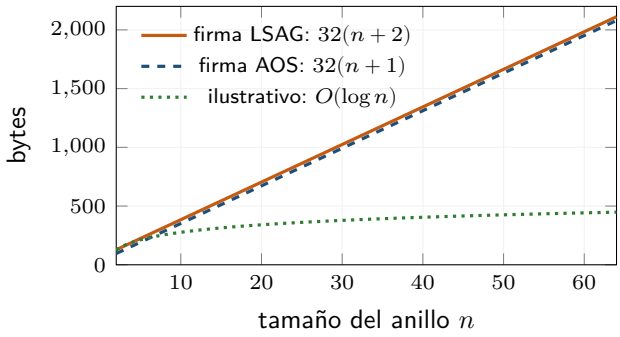
$$x_s = (r'_s - r_s)\,(c_s - c'_s)^{-1} \bmod \ell.$$

   La clave secreta queda expuesta. Se usa siempre un generador criptográfico, o una derivación determinista que mezcle x_s, m, L y aleatoriedad fresca.
2. **No incluir L en el hash**, o no fijar su orden: la firma deja de estar ligada a un anillo concreto.
3. **Aceptar puntos sin validar**: con Ed25519, un punto de orden pequeño puede romper la enlazabilidad o filtrar información.
4. **Comparar con `==`** cadenas secretas: se usa `secrets.compare_digest`.
5. **Usar un hash de 32 bytes reducido** directamente como escalar: produce desafíos ligeramente sesgados.

8 Etapa 10. Despliegue: el anonimato real

La matemática garantiza que la firma no favorece a ningún miembro. Pero el anonimato que percibe el usuario depende también de cómo se usa el sistema:

- **Composición del anillo.** Si los otros miembros son evidentemente falsos (claves recién creadas, cuentas inactivas, salidas muy antiguas), el anillo efectivo es más pequeño que n. Monero, por ejemplo, elige sus señuelos imitando la distribución de antigüedad de los gastos reales, después de que un estudio empírico mostrara que señuelos mal elegidos permitían rastrear muchas transacciones [8].
- **Metadatos.** La dirección IP, la hora de envío o el estilo de redacción pueden delatar al firmante aunque la firma sea perfecta.
- **Intersección.** Si una misma persona firma varias veces con anillos distintos, la intersección de los anillos puede reducir a los candidatos.
- **Computación cuántica.** El algoritmo de Shor rompe el logaritmo discreto. La enlazabilidad y la infalsificabilidad caerían; para el largo plazo se investigan firmas de anillo basadas en retículas, como MatRiCT [9].

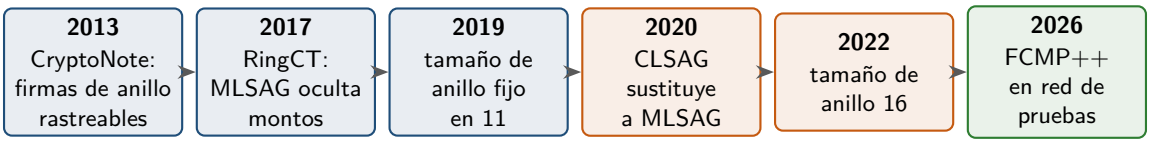


La curva $O(\log n)$ es solo ilustrativa: la constante real depende de la construcción.

9 Aplicaciones reales

Monero: la aplicación más extendida

Monero es el despliegue más grande de firmas de anillo. Cada vez que un usuario gasta una salida (un «billete» recibido antes), firma con un anillo que mezcla su salida real con salidas ajenas de la cadena como señuelos. Las claves de los señuelos se toman de transacciones pasadas, así que sus dueños no participan: es la propiedad *espontánea* de las firmas de anillo.



- **Evolución.** El protocolo CryptoNote [5] introdujo firmas de anillo rastreables con imagen de clave. RingCT [6] las combinó con montos ocultos mediante MLSAG. En 2020 se adoptó CLSAG [7], más compacta, y desde 2022 cada entrada se esconde entre 16 candidatos: la real y 15 señuelos.
- **Para qué sirve la imagen de clave.** Si alguien intenta gastar la misma salida dos veces, ambas firmas tendrían la misma I. Los nodos guardan todas las imágenes usadas y rechazan cualquier repetición. Es exactamente la función `enlazadas` de la sección 6.
- **Hacia dónde va.** El proyecto prepara FCMP++, que reemplaza el anillo de 16 por una prueba de pertenencia sobre todo el conjunto de salidas. Una red de pruebas de estrés con FCMP++ se bifurcó en octubre de 2026, sin fecha anunciada para la red principal [10]. Al escribir este documento, la red principal sigue usando CLSAG con anillos de 16.

Otras aplicaciones establecidas

- **Filtración de información.** El caso que motivó el concepto: un miembro de un gabinete prueba ante un periodista que la información viene de dentro, sin identificarse [1].
- **Autenticación negable.** Con un anillo de dos claves (la del emisor y la del receptor), el receptor queda convencido de quién escribió, porque él no lo hizo, pero no puede probarlo ante terceros: él mismo pudo haber producido la firma. Esta idea aparece ya en el artículo original [1].
- **Otras criptomonedas CryptoNote** heredaron el mismo mecanismo de firmas de anillo con imagen de clave.

10 Escenarios de uso

Los siguientes escenarios son plausibles con la tecnología actual; cada uno indica qué variante conviene y cuál es su riesgo principal.

Escenario	Quién forma el anillo	Variante	Riesgo principal
Votación electrónica	Padrón de votantes con claves registradas	LSAG	La imagen I detecta el doble voto; hay que impedir la coerción y la venta del voto.
Evaluación docente anónima	Alumnos inscritos en el grupo	LSAG por curso	Con grupos de 10 alumnos el anonimato es 1/10; el estilo del comentario puede delatar.
Canal de denuncias	Empleados o miembros de un comité	AOS	Metadatos de envío (IP, hora); anillos demasiado pequeños.
Revisión por pares	Comité de programa de un congreso	AOS	Se prueba que la revisión viene del comité sin revelar al revisor.
Consorcio de hospitales	Claves de los hospitales participantes	AOS o LSAG	Se prueba que un dato viene de un hospital del consorcio sin revelar cuál; hay que cuidar la calidad de los datos.
Credencial de pertenencia	Claves públicas de una institución, o SSH/PGP ya publicadas	AOS	Probar «soy alumno» o «soy desarrollador del proyecto»; las claves deben ser auténticas.
Gobernanza en una DAO	Titulares de fichas de membresía	LSAG	Una persona con varias claves vota varias veces.
Redes vehiculares o IoT	Dispositivos certificados de una zona	LSAG	Restricciones de tiempo real y de tamaño de mensaje.

Cómo decidir si un problema necesita una firma de anillo

Hazte tres preguntas. **(1)** ¿Hay que probar que algo viene de *un grupo* sin decir de quién? **(2)** ¿Existen ya claves públicas auténticas de los miembros? **(3)** ¿Es aceptable que nadie, ni siquiera un administrador, pueda revelar al firmante? Si las tres respuestas son «sí», una firma de anillo es la herramienta natural. Si hace falta poder revelar al firmante en caso de abuso, lo indicado es una *firma de grupo*. Si además debe limitarse el número de firmas por persona, conviene la variante enlazable.

11 Lista de verificación final

- ☐ Grupo de orden primo de unos 256 bits; biblioteca auditada.
- ☐ Hash con separación de dominio, codificación inyectiva y reducción de 64 bytes.
- ☐ L, m (y en LSAG, I) dentro de cada hash; anillo en orden canónico.
- ☐ Aleatoriedad criptográfica nueva en cada firma; nunca reutilizar a.
- ☐ Validación de todos los puntos y escalares recibidos.
- ☐ Comparaciones en tiempo constante.
- ☐ Pruebas de ida y vuelta, manipulación, vectores fijos y entradas hostiles.
- ☐ Anillos con señuelos creíbles y atención a los metadatos.

Referencias

[1] R. L. Rivest, A. Shamir y Y. Tauman. *How to Leak a Secret*. ASIACRYPT 2001, LNCS 2248, Springer, 2001.

[2] M. Abe, M. Ohkubo y K. Suzuki. *1-out-of-n Signatures from a Variety of Keys*. ASIACRYPT 2002, LNCS 2501, Springer, 2002.

[3] J. K. Liu, V. K. Wei y D. S. Wong. *Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups*. ACISP 2004, LNCS 3108, Springer, 2004.

[4] J. Groth y M. Kohlweiss. *One-out-of-Many Proofs: Or How to Leak a Secret and Spend a Coin*. EUROCRYPT 2015, LNCS 9057, Springer, 2015.

[5] N. van Saberhagen. *CryptoNote v2.0*. Libro blanco, 2013.

[6] S. Noether. *Ring Confidential Transactions*. Ledger, vol. 1, 2016.

[7] B. Goodell, S. Noether y A. Blue (RandomRun). *Concise Linkable Ring Signatures and Forgery Against Adversarial Keys*. IACR Cryptology ePrint Archive 2019/654.

[8] M. Möser et al. *An Empirical Analysis of Traceability in the Monero Blockchain*. Proceedings on Privacy Enhancing Technologies, 2018(3).

[9] M. F. Esgin, R. K. Zhao, R. Steinfeld, J. K. Liu y D. Liu. *MatRiCT: Efficient, Scalable and Post-Quantum Blockchain Confidential Transactions Protocol*. ACM CCS 2019.

[10] CryptoTicker. *Monero FCMP++: testnet fork on October 5*. Octubre de 2026. `https://cryptoticker.io/en/monero-fcmp-stressnet-fork-october-2026/`

[11] F. Denis et al. *Documentación de libsodium*, secciones sobre operaciones de bajo nivel en Ed25519. `https://doc.libsodium.org`